Exceptions - exception hierarchy - throwing and catching exceptions built-in exceptions, creating own exceptions, Stack Trace Elements. Input / Output Basics Streams Byte streams and Character streams Reading and Writing Console Reading and Writing Files.

Exception Handling in Java

1. Exception Handling
2. Advantage of Exception Handling
3. Hierarchy of Exception classes
4. Types of Exception
5. Exception Example
6. Scenarios where an exception may occur

The **Exception Handling in Java** is one of e application can be maintained.

What is Exception?

Exception is an abnormal condition. In Java, an exception is an event that disrupts the normal flow of the program. It is an object which is thrown at runtime.

Exception Handling is a mechanism to handle runtime errors such as ClassNotFoundException, IOException, SQLException, RemoteException, etc.

Advantage of Exception Handling

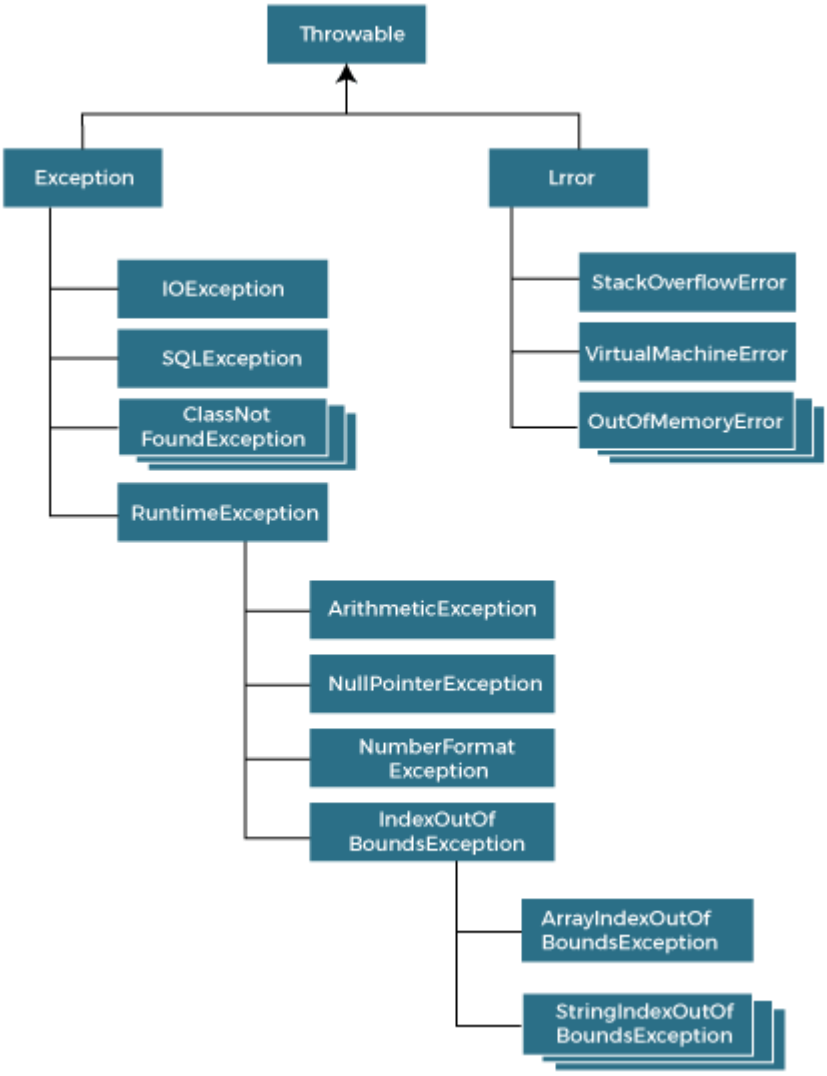
The core advantage of exception handling is **to maintain the normal flow of the application**. An exception normally disrupts the normal flow of the application; that is why we need to handle exceptions. Let's consider a scenario:

```
1. statement 1;
2. statement 2;
3. statement 3;
4. statement 4;
5. statement 5;//exception occurs
6. statement 6;
7. statement 7;
8. statement 8;
9. statement 9;
10. statement 10;
```

Suppose there are 10 statements in a Java program and an exception occurs at statement 5; the rest of the code will not be executed, i.e., statements 6 to 10 will not be executed. However, when we perform exception handling, the rest of the statements will be executed. That is why we use exception handling in Java.

Hierarchy of Java Exception classes

The java.lang.Throwable class is the root class of Java Exception hierarchy inherited by two subclasses: Exception and Error. The hierarchy of Java Exception classes is given below:



Types of Java Exceptions

There are mainly two types of exceptions: checked and unchecked. An error is considered as the unchecked exception. However, according to Oracle, there are three types of exceptions namely:

1. Checked Exception
2. Unchecked Exception
3. Error

Difference between Checked and Unchecked Exceptions

1) Checked Exception

The classes that directly inherit the Throwable class except RuntimeException and Error are known as checked exceptions. For example, IOException, SQLException, etc. Checked exceptions are checked at compile-time.

2) Unchecked Exception

The classes that inherit the RuntimeException are known as unchecked exceptions. For example, ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, etc. Unchecked exceptions are not checked at compile-time, but they are checked at runtime.

3) Error

Error is irrecoverable. Some example of errors are OutOfMemoryError, VirtualMachineError, AssertionError etc.

Java Exception Keywords

Java provides five keywords that are used to handle the exception. The following table describes each.

Keyword	Description
try	The "try" keyword is used to specify a block where we should place an exception code. It means we can't use try block alone. The try block must be followed by either catch or finally.
catch	The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.
finally	The "finally" block is used to execute the necessary code of the

	program. It is executed whether an exception is handled or not.
throw	The "throw" keyword is used to throw an exception.
throws	The "throws" keyword is used to declare exceptions. It specifies that there may occur an exception in the method. It doesn't throw an exception. It is always used with method signature

Java Exception Handling Example

Let's see an example of Java Exception Handling in which we are using a try-catch statement to handle the exception.

JavaExceptionExample.java

```
1. public class JavaExceptionExample{
2.   public static void main(String args[]){
3.    try{
4.      //code that may raise exception
5.      int data=100/0;
6.    }
7. catch(ArithmeticException e)
8. {
9. System.out.println(e);
10. }
11.   //rest code of the program
12.   System.out.println("Welcome to the code...");
13.  }
14. }
```

Output:

```
Exception in thread main java.lang.ArithmeticException:/ by zero
Welcome to the code...
```

In the above example, 100/0 raises an ArithmeticException which is handled by a try-catch block.

Common Scenarios of Java Exceptions

There are given some scenarios where unchecked exceptions may occur. They are as follows:

1) A scenario where ArithmeticException occurs

If we divide any number by zero, there occurs an ArithmeticException.

```
int a=50/0;//ArithmeticException
```

2) A scenario where NullPointerException occurs

If we have a null value in any variable, performing any operation on the variable throws a NullPointerException.

```
String s=null;
System.out.println(s.length());//NullPointerException
```

3) A scenario where NumberFormatException occurs

If the formatting of any variable or number is mismatched, it may result into NumberFormatException. Suppose we have a string variable that has characters; converting this variable into digit will cause NumberFormatException.

```
String s="abc";
int i=Integer.parseInt(s);//NumberFormatException
```

4) A scenario where ArrayIndexOutOfBoundsException occurs

When an array exceeds to it's size, the ArrayIndexOutOfBoundsException occurs. there may be other reasons to occur ArrayIndexOutOfBoundsException. Consider the following statements.

```
int a[]=new int[5];
a[10]=50; //ArrayIndexOutOfBoundsException
```

Java try-catch block:

Java try block

Java **try** block is used to enclose the code that might throw an exception. It must be used within the method.

If an exception occurs at the particular statement in the try block, the rest of the block code will not execute. So, it is recommended not to keep the code in try block that will not throw an exception.

Java try block must be followed by either catch or finally block.

Syntax of Java try-catch

```
1. try{
2. //code that may throw an exception
3. }catch(Exception_class_Name ref){}
```

Syntax of try-finally block

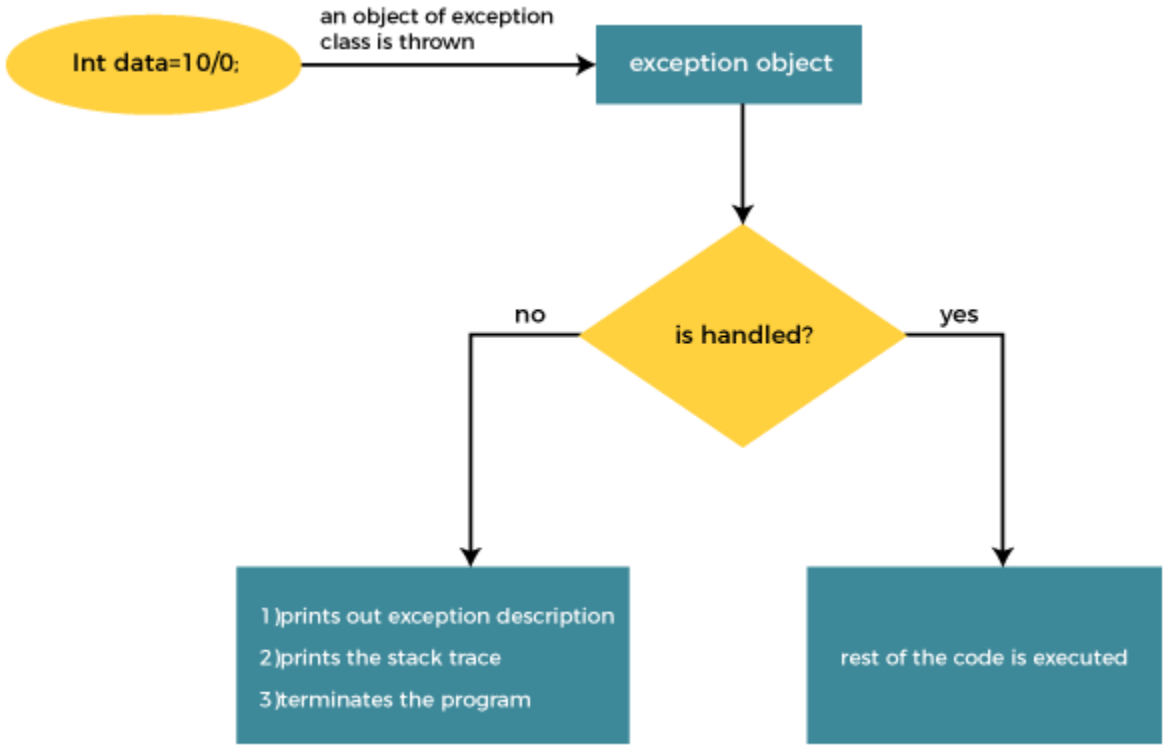
```
1. try{
2. //code that may throw an exception
3. }finally{}
```

Java catch block

Java catch block is used to handle the Exception by declaring the type of exception within the parameter. The declared exception must be the parent class exception (i.e., Exception) or the generated exception type. However, the good approach is to declare the generated type of exception.

The catch block must be used after the try block only. You can use multiple catch block with a single try block.

Internal Working of Java try-catch block



The JVM firstly checks whether the exception is handled or not. If exception is not handled, JVM provides a default exception handler that performs the following tasks:

- Prints out exception description.
- Prints the stack trace (Hierarchy of methods where the exception occurred).
- Causes the program to terminate.

But if the application programmer handles the exception, the normal flow of the application is maintained, i.e., rest of the code is executed.

Problem without exception handling

Let's try to understand the problem if we don't use a try-catch block.

Example 1

TryCatchExample1.java

```
1. public class TryCatchExample1 {
2.
3.    public static void main(String[] args) {
4.
5.       int data=50/0; //may throw exception
6.
7.       System.out.println("rest of the code");
8.
9.    }
10.
11. }
```

Output:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
```

As displayed in the above example, the **rest of the code** is not executed (in such case, the **rest of the code** statement is not printed).

There might be 100 lines of code after the exception. If the exception is not handled, all the code below the exception won't be executed.

Solution by exception handling

Let's see the solution of the above problem by a java try-catch block.

Example 2

TryCatchExample2.java

```
public class TryCatchExample2 {
```

```
public static void main(String[] args) {
  try
  {
  int data=50/0; //may throw exception
  }
     //handling the exception
  catch(ArithmeticException e)
  {
     System.out.println(e);
  }
  System.out.println("rest of the code");
     }

     }
```

Output:

```
java.lang.ArithmeticException: / by zero
rest of the code
```

As displayed in the above example, the **rest of the code** is executed, i.e., the **rest of the code** statement is printed.

Example 3

In this example, we also kept the code in a try block that will not throw an exception.

TryCatchExample3.java

```
1.  public class TryCatchExample3 {
2.
3.    public static void main(String[] args) {
4.       try
5.       {
6.       int data=50/0; //may throw exception
7.                    // if exception occurs, the remaining statement will not exceute
8.       System.out.println("rest of the code");
9.       }
10.          // handling the exception
11.      catch(ArithmeticException e)
12.      {
13.         System.out.println(e);
```

```
14.         }
15.
16.     }
17.
18. }
```

Output:

```
java.lang.ArithmeticException: / by zero
```

Example 5

Let's see an example to print a custom message on exception.

TryCatchExample5.java

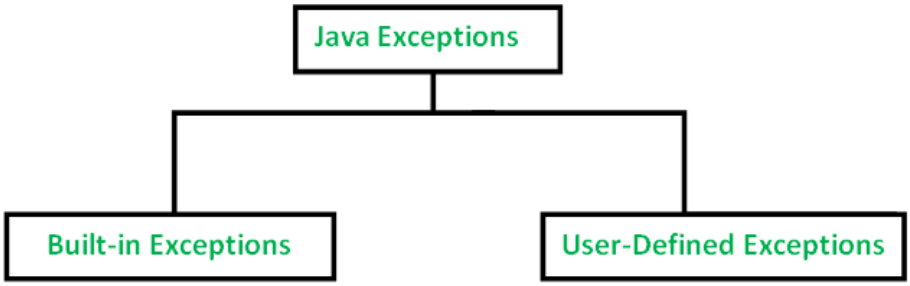
```
1.  public class TryCatchExample5 {
2.
3.      public static void main(String[] args) {
4.          try
5.          {
6.          int data=50/0; //may throw exception
7.          }
8.              // handling the exception
9.          catch(Exception e)
10.         {
11.                 // displaying the custom message
12.             System.out.println("Can't divided by zero");
13.         }
14.     }
15.
16. }
```

Output:

```
Can't divided by zero
```

Types of Exception in Java with Examples

Java defines several types of exceptions that relate to its various class libraries. Java also allows users to define their own exceptions.



Built-in Exceptions

Built-in exceptions are the exceptions which are available in Java libraries. These exceptions are suitable to explain certain error situations. Below is the list of important built-in exceptions in Java.

1. **ArithmeticException**

   It is thrown when an exceptional condition has occurred in an arithmetic operation.

2. **ArrayIndexOutOfBoundsException**

   It is thrown to indicate that an array has been accessed with an illegal index. The index is either negative or greater than or equal to the size of the array.

3. **ClassNotFoundException**

   This Exception is raised when we try to access a class whose definition is not found

4. **FileNotFoundException**

   This Exception is raised when a file is not accessible or does not open.

5. **IOException**

   It is thrown when an input-output operation failed or interrupted

6. **InterruptedException**

   It is thrown when a thread is waiting, sleeping, or doing some processing, and it is interrupted.

7. **NoSuchFieldException**

   It is thrown when a class does not contain the field (or variable) specified

8. **NoSuchMethodException**

   It is thrown when accessing a method which is not found.

9. **NullPointerException**

   This exception is raised when referring to the members of a null object. Null represents nothing

10. **NumberFormatException**

    This exception is raised when a method could not convert a string into a numeric format.

11. **RuntimeException**

    This represents any exception which occurs during runtime.

12. **StringIndexOutOfBoundsException**

    It is thrown by String class methods to indicate that an index is either negative or greater than the size of the string

Examples of Built-in Exception:

Arithmetic exception:

```
// Java program to demonstrate ArithmeticException
class ArithmeticException_Demo
{
   public static void main(String args[])
   {
      try {
         int a = 30, b = 0;
         int c = a/b;  // cannot divide by zero
         System.out.println ("Result = " + c);
      }
      catch(ArithmeticException e) {
         System.out.println ("Can't divide a number by 0");
      }
   }
}
```

Output:

Can't divide a number by 0

Write a program to get two numbers and divide then print the result.. using Exception Handling for divide by zero error:

NullPointer Exception:

```
//Java program to demonstrate NullPointerException
class NullPointer_Demo
{
   public static void main(String args[])
   {
      try {
         String a = null; //null value
         System.out.println(a.charAt(0));
      } catch(NullPointerException e) {
         System.out.println("NullPointerException..");
      }
   }
}
```

Output:

NullPointerException..

StringIndexOutOfBound Exception:

```
// Java program to demonstrate StringIndexOutOfBoundsException
class StringIndexOutOfBound_Demo
{
   public static void main(String args[])
   {
      try {
         String a = "This is like chipping "; // length is 22
         char c = a.charAt(24); // accessing 25th element
         System.out.println(c);
      }
      catch(StringIndexOutOfBoundsException e) {
         System.out.println("StringIndexOutOfBoundsException");
      }
   }
}
```

Output:

StringIndexOutOfBoundsException

FileNotFound Exception:

```
//Java program to demonstrate FileNotFoundException
import java.io.File;
import java.io.FileNotFoundException;
import java.io.FileReader;
 class File_notFound_Demo {

   public static void main(String args[])  {
      try {

         // Following file does not exist
         File file = new File("E://file.txt");

         FileReader fr = new FileReader(file);
      } catch (FileNotFoundException e) {
        System.out.println("File does not exist");
      }
   }
}
```

Output:

File does not exist

NumberFormat Exception

```
// Java program to demonstrate NumberFormatException
class  NumberFormat_Demo
{
   public static void main(String args[])
   {
      try {
         // "akki" is not a number
         int num = Integer.parseInt ("akki") ;

         System.out.println(num);
      } catch(NumberFormatException e) {
         System.out.println("Number format exception");
      }
   }
}
```

Output:

Number format exception

ArrayIndexOutOfBounds Exception:

```
// Java program to demonstrate ArrayIndexOutOfBoundException
class ArrayIndexOutOfBound_Demo
{
   public static void main(String args[])
   {
      try{
         int a[] = new int[5];
         a[6] = 9; // accessing 7th element in an array of
                  // size 5
      }
      catch(ArrayIndexOutOfBoundsException e){
         System.out.println ("Array Index is Out Of Bounds");
      }
   }
}
```

Output:
Array Index is Out Of Bounds

User-Defined Exceptions:
Sometimes, the built-in exceptions in Java are not able to describe a certain situation. In such cases, user can also create exceptions which are called 'user-defined Exceptions'.
Following steps are followed for the creation of user-defined Exception.

- The user should create an exception class as a subclass of Exception class. Since all the exceptions are subclasses of Exception class, the user should also make his class a subclass of it.

  This is done as:
  class MyException extends Exception
- We can write a default constructor in his own exception class.
  MyException(){}

To raise exception of user-defined type, we need to create an object to his exception class and throw it using throw clause, as:

MyException me = new MyException("Exception details");

throw me;

- The following program illustrates how to create own exception class MyException.
- Details of account numbers, customer names, and balance amounts are taken in the form of three arrays.
- In main() method, the details are displayed using a for-loop. At this time, check is done if in any account the balance amount is less than the minimum balance amount to be apt in the account.
- If it is so, then MyException is raised and a message is displayed "Balance amount is less".

Java program to demonstrate user defined exception

```
// This program throws an exception whenever balance
// amount is below Rs 1000
public class MyException extends Exception
{
   //store account information
   private static int accno[] = {1001, 1002, 1003, 1004};

   private static String name[] =
            {"Nish", "Shubh", "Sush", "Abhi", "Akash"};

   private static double bal[] =
       {10000.00, 12000.00, 5600.0, 999.00, 1100.55};

   // default constructor
   MyException() {    }

   // parameterized constructor
   MyException(String str) { super(str); }

   // write main()
   public static void main(String[] args)
   {
      try  {
         // display the heading for the table
         System.out.println("ACCNO" + "\t" + "CUSTOMER" +
                              "\t" + "BALANCE");

         // display the actual account information
         for (int i = 0; i < 5 ; i++)
         {
            System.out.println(accno[i] + "\t" + name[i] +
                                 "\t" + bal[i]);

            // display own exception if balance < 1000
```

```
            if (bal[i] < 1000)
            {
               MyException me =
                 new MyException(name[i]+" " +"Balance is less than 1000");
                 throw me;
            }
         }
      }//end of try

      catch (MyException e) {
         e.printStackTrace();
      }
   }
}
```

```
$javac MyException.java
$java -Xmx128M -Xms16M MyException
ACCNO          CUSTOMER     BALANCE
1001     Nish          10000.0
1002     Shubh         12000.0
1003     Sush          5600.0
1004     Abhi          999.0
MyException: Abhi Balance is less than 1000
         at MyException.main(MyException.java:35)
```

Write a java program to get details of students along with marks, and chk the marks which is less than 50 using user defined Exception handling method. (Get the data at runtime).

Java Catch Multiple Exceptions

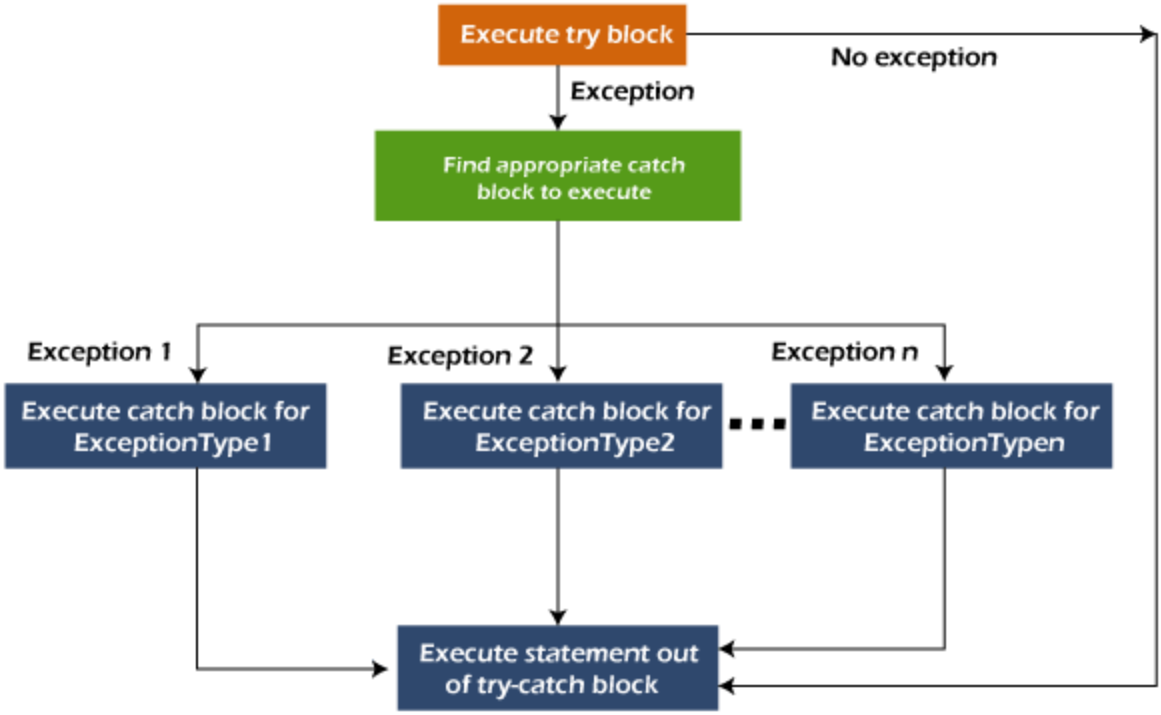
Java Multi-catch block

A try block can be followed by one or more catch blocks. Each catch block must contain a different exception handler. So, if you have to perform different tasks at the occurrence of different exceptions, use java multi-catch block.

Points to remember

- At a time only one exception occurs and at a time only one catch block is executed.
- All catch blocks must be ordered from most specific to most general, i.e. catch for ArithmeticException must come before catch for Exception.

Flowchart of Multi-catch Block



Example 1

Let's see a simple example of java multi-catch block.

MultipleCatchBlock1.java

```
1.  public class MultipleCatchBlock1 {
2.
3.     public static void main(String[] args) {
4.
5.          try{
6.              int a[]=new int[5];
7.
8.             a[5]=30/0;
9.             }
10.            catch(ArithmeticException e)
11.              {
12.               System.out.println("Arithmetic Exception occurs");
13.              }
14.            catch(ArrayIndexOutOfBoundsException e)
15.              {
16.               System.out.println("ArrayIndexOutOfBounds Exception occurs");
17.              }
18.            catch(Exception e)
19.              {
20.               System.out.println("Parent Exception occurs");
21.              }
22.            System.out.println("rest of the code");
23.    }
24. }
```

Test it Now

Output:

```
Arithmetic Exception occurs
rest of the code
```

Example 2

MultipleCatchBlock2.java

```
1.  public class MultipleCatchBlock2 {
2.
3.     public static void main(String[] args) {
4.
5.          try{
6.              int a[]=new int[5];
7.
8.              System.out.println(a[10]);
9.             }
10.            catch(ArithmeticException e)
11.              {
12.               System.out.println("Arithmetic Exception occurs");
13.              }
14.            catch(ArrayIndexOutOfBoundsException e)
15.              {
16.               System.out.println("ArrayIndexOutOfBounds Exception occurs");
17.              }
18.            catch(Exception e)
19.              {
20.               System.out.println("Parent Exception occurs");
21.              }
22.            System.out.println("rest of the code");
23.    }
24. }
```

Test it Now

Output:

```
ArrayIndexOutOfBounds Exception occurs
rest of the code
```

In this example, try block contains two exceptions. But at a time only one exception occurs and its corresponding catch block is executed.

MultipleCatchBlock3.java

```
1.  public class MultipleCatchBlock3 {
2.
3.     public static void main(String[] args) {
4.
5.          try{
6.              int a[]=new int[5];
7.              a[5]=30/0;
8.              System.out.println(a[10]);
9.             }
10.            catch(ArithmeticException e)
11.              {
12.               System.out.println("Arithmetic Exception occurs");
13.              }
14.            catch(ArrayIndexOutOfBoundsException e)
15.              {
16.               System.out.println("ArrayIndexOutOfBounds Exception occurs");
17.              }
18.            catch(Exception e)
19.              {
20.               System.out.println("Parent Exception occurs");
21.              }
22.            System.out.println("rest of the code");
23.    }
24. }
```

Test it Now

Output:

```
Arithmetic Exception occurs
```

```
rest of the code
```

Example 4

In this example, we generate NullPointerException, but didn't provide the corresponding exception type. In such case, the catch block containing the parent exception class **Exception** will invoked.

MultipleCatchBlock4.java

```
1.  public class MultipleCatchBlock4 {
2.
3.     public static void main(String[] args) {
4.
5.          try{
6.              String s=null;
7.              System.out.println(s.length());
8.             }
9.             catch(ArithmeticException e)
10.              {
11.               System.out.println("Arithmetic Exception occurs");
12.              }
13.            catch(ArrayIndexOutOfBoundsException e)
14.              {
15.               System.out.println("ArrayIndexOutOfBounds Exception occurs");
16.              }
17.            catch(Exception e)
18.              {
19.               System.out.println("Parent Exception occurs");
20.              }
21.            System.out.println("rest of the code");
22.    }
23. }
```

Test it Now

Output:

```
Parent Exception occurs
rest of the code
```

Example 5

Let's see an example, to handle the exception without maintaining the order of exceptions (i.e. from most specific to most general).

MultipleCatchBlock5.java

```
1. class MultipleCatchBlock5{
2.   public static void main(String args[]){
3.    try{
4.     int a[]=new int[5];
5.     a[5]=30/0;
6.    }
7.    catch(Exception e){System.out.println("common task completed");}
8.    catch(ArithmeticException e){System.out.println("task1 is completed");}
9.    catch(ArrayIndexOutOfBoundsException e){System.out.println("task 2 completed");}

10.   System.out.println("rest of the code...");
11.  }
12. }
```

Test it Now

Output:

```
Compile-time error
```

Java throw Exception

In Java, exceptions allows us to write good quality codes where the errors are checked at the compile time instead of runtime and we can create custom exceptions making the code recovery and debugging easier.

Java throw keyword

The Java throw keyword is used to throw an exception explicitly.

We specify the **exception** object which is to be thrown. The Exception has some message with it that provides the error description. These exceptions may be related to user inputs, server, etc.

We can throw either checked or unchecked exceptions in Java by throw keyword. It is mainly used to throw a custom exception.

We can also define our own set of conditions and throw an exception explicitly using throw keyword. For example, we can throw ArithmeticException if we divide a number by another number. Here, we just need to set the condition and throw exception using throw keyword.

The syntax of the Java throw keyword is given below.

throw Instance i.e.,

```
1. throw new exception_class("error message");
```

Let's see the example of throw IOException.

```
1. throw new IOException("sorry device error");
```

Where the Instance must be of type Throwable or subclass of Throwable. For example, Exception is the sub class of Throwable and the user-defined exceptions usually extend the Exception class.

Java throw keyword Example

Example 1: Throwing Unchecked Exception

In this example, we have created a method named validate() that accepts an integer as a parameter. If the age is less than 18, we are throwing the ArithmeticException otherwise print a message welcome to vote.

TestThrow1.java

In this example, we have created the validate method that takes integer value as a parameter. If the age is less than 18, we are throwing the ArithmeticException otherwise print a message welcome to vote.

```
1.  public class TestThrow1 {
2.     //function to check if person is eligible to vote or not
3.     public static void validate(int age) {
4.        if(age<18) {
5.           //throw Arithmetic exception if not eligible to vote
6.           throw new ArithmeticException("Person is not eligible to vote");
7.        }
8.        else {
9.           System.out.println("Person is eligible to vote!!");
10.       }
11.    }
12.    //main method
13.    public static void main(String args[]){
14.       //calling the function
15.       validate(13);
16.       System.out.println("rest of the code...");
17.  }
18. }
```

Output:

```
C:\Users\Anurati\Desktop\abcDemo>javac TestThrow1.java

C:\Users\Anurati\Desktop\abcDemo>java TestThrow1
Exception in thread "main" java.lang.ArithmeticException: Person is not eligible to
 vote
        at TestThrow1.validate(TestThrow1.java:8)
        at TestThrow1.main(TestThrow1.java:18)
```

Java I/O Tutorial

Java I/O (Input and Output) is used *to process the input* and *produce the output*.

Java uses the concept of a stream to make I/O operation fast. The java.io package contains all the classes required for input and output operations.

We can perform **file handling in Java** by Java I/O API.

Stream

A stream is a sequence of data. In Java, a stream is composed of bytes. It's called a stream because it is like a stream of water that continues to flow.

In Java, 3 streams are created for us automatically. All these streams are attached with the console.

1) System.out: standard output stream

2) System.in: standard input stream

3) System.err: standard error stream

Let's see the code to print **output and an error** message to the console.

```
1. System.out.println("simple message");
2. System.err.println("error message");
```

Let's see the code to get **input** from console.

```
1. int i=System.in.read();//returns ASCII code of 1st character
2. System.out.println((char)i);//will print the character
```

OutputStream vs InputStream

The explanation of OutputStream and InputStream classes are given below:

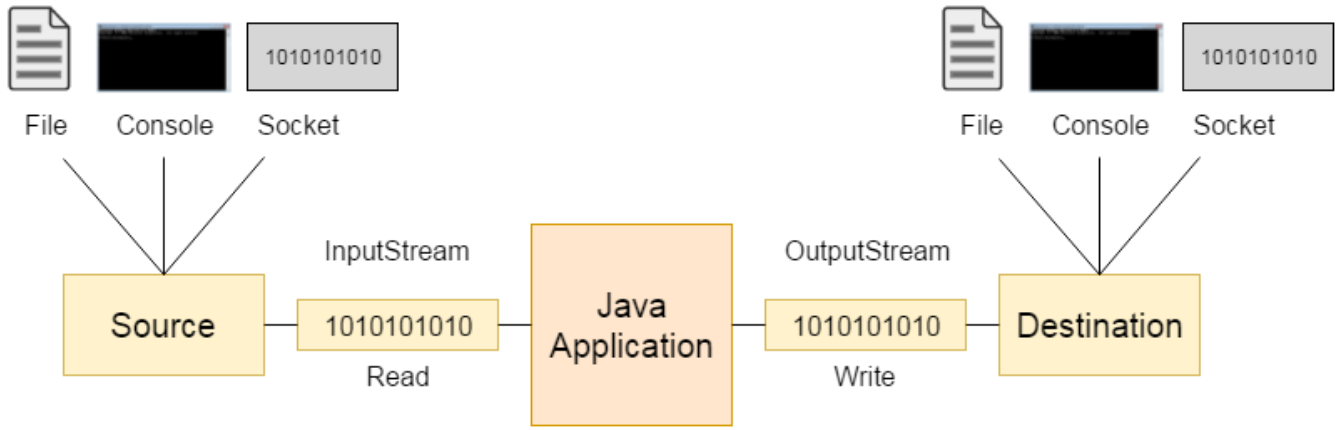
OutputStream

Java application uses an output stream to write data to a destination; it may be a file, an array, peripheral device or socket.

InputStream

Java application uses an input stream to read data from a source; it may be a file, an array, peripheral device or socket.

Let's understand the working of Java OutputStream and InputStream by the figure given below.

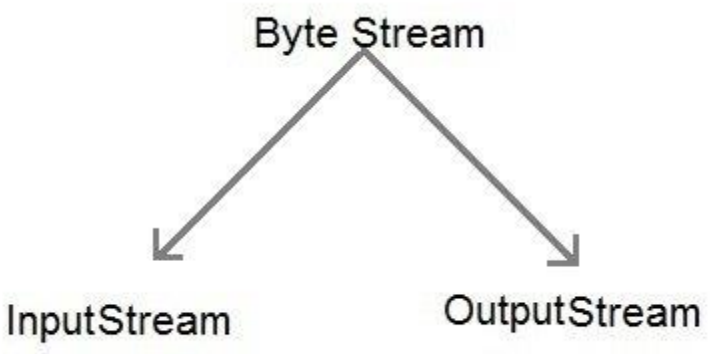


Java encapsulates Stream under **java.io** package. Java defines two types of streams. They are,

1. **Byte Stream :** It provides a convenient means for handling input and output of byte.
2. **Character Stream :** It provides a convenient means for handling input and output of characters. Character stream uses Unicode and therefore can be internationalized.

Java Byte Stream Classes

Byte stream is defined by using two abstract class at the top of hierarchy, they are InputStream and OutputStream.



These two abstract classes have several concrete classes that handle various devices such as disk files, network connection etc.

Some important Byte stream classes.

Stream class	Description
BufferedInputStream	Used for Buffered Input Stream.
BufferedOutputStream	Used for Buffered Output Stream.
DataInputStream	Contains method for reading java standard datatype
DataOutputStream	An output stream that contain method for writing java standard data type

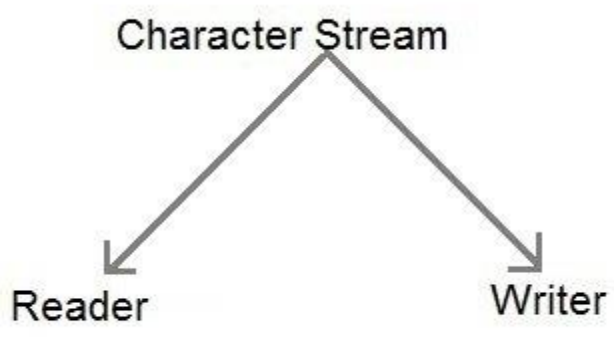
FileInputStream	Input stream that reads from a file
FileOutputStream	Output stream that write to a file.
InputStream	Abstract class that describe stream input.
OutputStream	Abstract class that describe stream output.
PrintStream	Output Stream that contain print() and println() method

These classes define several key methods. Two most important are

1. read() : reads byte of data.
2. write() : Writes byte of data.

Java Character Stream Classes

Character stream is also defined by using two abstract class at the top of hierarchy, they are Reader and Writer.



These two abstract classes have several concrete classes that handle unicode character.

Some important Charcter stream classes

Stream class	Description
BufferedReader	Handles buffered input stream.
BufferedWriter	Handles buffered output stream.
FileReader	Input stream that reads from file.
FileWriter	Output stream that writes to file.
InputStreamReader	Input stream that translate byte to character
OutputStreamReader	Output stream that translate character to byte.
PrintWriter	Output Stream that contain print() and println() method.
Reader	Abstract class that define character stream input
Writer	Abstract class that define character stream output

Java File Handling

The File class from the java.io package, allows us to work with files.

To use the File class, create an object of the class, and specify the filename or directory name:

Example

```
import java.io.File;  // Import the File class

File myObj = new File("filename.txt");

File myobj1 = new File("file2.ppt");

 // Specify the filename
```

The File class has many useful methods for creating and getting information about files. For example:

Method	Return Type	Description
canRead()	Boolean	Tests whether the file is readable or not
canWrite()	Boolean	Tests whether the file is writable or not
createNewFile()	Boolean	Creates an empty file
delete()	Boolean	Deletes a file
exists()	Boolean	Tests whether the file exists

getName()	String	Returns the name of the file
getAbsolutePath()	String	Returns the absolute pathname of the file
length()	Long	Returns the size of the file in bytes
list()	String[]	Returns an array of the files in the directory
mkdir()	Boolean	Creates a directory

Create a File

To create a file in Java, you can use the createNewFile() method. This method returns a boolean value: true if the file was successfully created, and false if the file already exists. Note that the method is enclosed in a try...catch block. This is necessary because it throws an IOException if an error occurs (if the file cannot be created for some reason):

Example

```
import java.io.File;  // Import the File class
import java.io.IOException;  // Import the IOException class to handle errors

public class CreateFile {
  public static void main(String[] args) {
    try {
      File myObj = new File("filename.txt");
      if (myObj.createNewFile()) {
        System.out.println("File created: " + myObj.getName());
      } else {
        System.out.println("File already exists.");
      }
    } catch (IOException e) {
      System.out.println("An error occurred.");
      e.printStackTrace();
    }
  }
}
```

The output will be:

File created: filename.txt

Run Example »

To create a file in a specific directory (requires permission), specify the path of the file and use double backslashes to escape the "\" character (for Windows). On Mac and Linux you can just write the path, like: /Users/name/filename.txt

Example

```
File myObj = new File("C:\\Users\\MyName\\filename.txt");
```

Write To a File

In the following example, we use the FileWriter class together with its write() method to write some text to the file we created in the example above. Note that when you are done writing to the file, you should close it with the close() method:

Example

```
import java.io.FileWriter;   // Import the FileWriter class
import java.io.IOException;  // Import the IOException class to handle errors

public class WriteToFile {
  public static void main(String[] args) {
    try {
      FileWriter myWriter = new FileWriter("filename.txt");
      myWriter.write("Files in Java might be tricky, but it is fun enough!");
      myWriter.close();
      System.out.println("Successfully wrote to the file.");
    } catch (IOException e) {
      System.out.println("An error occurred.");
      e.printStackTrace();
    }
  }
}
```

The output will be:

Successfully wrote to the file.

Get File Information

The operation is performed to get the file information. We use several methods to get the information about the file like name, absolute path, is readable, is writable and length.

Let's take an example to understand how to use file methods to get the information of the file.

FileInfo.java

```
1. // Import the File class
2. import java.io.File;
3. class FileInfo {
4. public static void main(String[] args) {
5. // Creating file object
6. File f0 = new File("D://FileOperationExample.txt");
7. if (f0.exists()) {
8. // Getting file name
9. System.out.println("The name of the file is: " + f0.getName());

10. // Getting path of the file
11. System.out.println("The absolute path of the file is: " + f0.getAbsolutePath());

12. // Checking whether the file is writable or not
13. System.out.println("Is file writeable?: " + f0.canWrite());

14. // Checking whether the file is readable or not
15. System.out.println("Is file readable " + f0.canRead());

16. // Getting the length of the file in bytes
17. System.out.println("The size of the file in bytes is: " + f0.length());
18. } else {
19. System.out.println("The file does not exist.");
20. }
21. }
22. }
```

```
C:\Windows\System32\cmd.exe

C:\Users\ajeet\OneDrive\Desktop\programs>javac FileInfo.java

C:\Users\ajeet\OneDrive\Desktop\programs>java FileInfo
The name of the file is: FileOperationExample.txt
The absolute path of the file is: D:\\FileOperationExample.txt
Is file writeable?: true
Is file readable true
The size of the file in bytes is: 0

C:\Users\ajeet\OneDrive\Desktop\programs>
```

Description:

In the above code, we import the **java.io.File** package and create a class **FileInfo**. In the main method, we create an object of the text file which we have created in our previous example. We check the existence of the file using a conditional statement, and if it is present, we get the following information about that file:

1. We get the name of the file using the **getName()**
2. We get the absolute path of the file using the **getAbsolutePath()** method of the file.
3. We check whether we can write data into a file or not using the **canWrite()**
4. We check whether we can read the data of the file or not using the **canRead()**
5. We get the length of the file by using the **length()**

Read from a File

The next operation which we can perform on a file is **"read from a file"**. In order to write data into a file, we will use the **Scanner** class. Here, we need to close the stream using the **close()** method. We will create an instance of the Scanner class and use the **hasNextLine()** method **nextLine()** method to get data from the file.

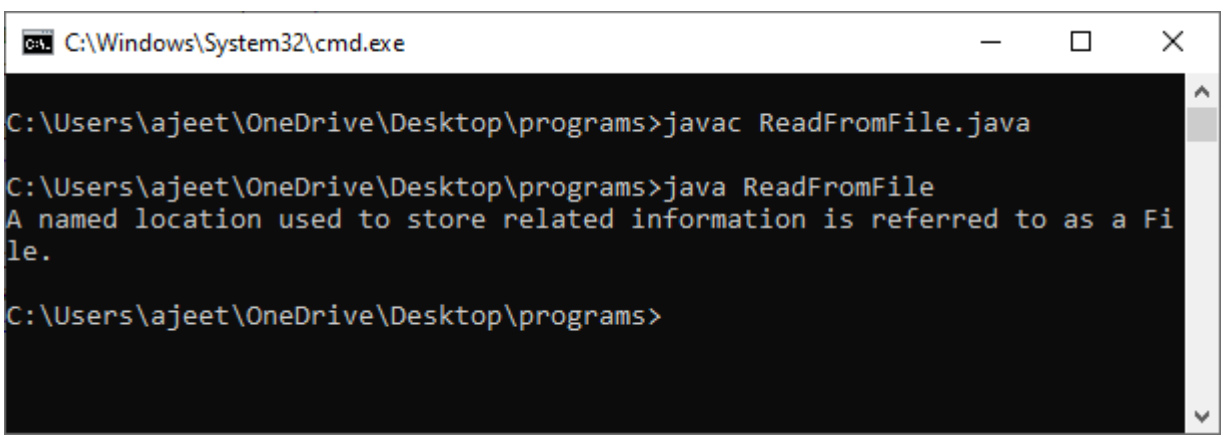
Let's take an example to understand how we can read data from a file.

ReadFromFile.java

```
// Importing the File class
import java.io.File;
// Importing FileNotFoundException class for handling errors
import java.io.FileNotFoundException;
// Importing the Scanner class for reading text files
import java.util.Scanner;

class ReadFromFile {
  public static void main(String[] args) {
    try {
      // Create f1 object of the file to read data
      File f1 = new File("D://FileOperationExample.txt");
      Scanner dataReader = new Scanner(f1);
      while (dataReader.hasNextLine()) {
        String fileData = dataReader.nextLine();
        System.out.println(fileData);
      }
      dataReader.close();
    } catch (FileNotFoundException exception) {
      System.out.println("Unexcpected error occurred!");
      exception.printStackTrace();
    }
  }
}
```

Output:



Expalnation:

In the above code, we import the **"java.util.Scannner", "java.io.File"** and **"java.io.IOException"** classes. We create a class **ReadFromFile**, and in its main method, we use the **try-catch block**. In the try section, we create an instance of both the **Scanner** and the **File** classes. We pass the **File** class object to the **Scanner** class object and then iterate the scanner class object using the **"While"** loop and print each line of the file. We also need to close the scanner class object, so we use the close() function. If we get any error in the try section, it jumps to the catch block. In the catch block, we handle the IOException and print a custom message.

Delete a File

The next operation which we can perform on a file is **"deleting a file"**. In order to delete a file, we will use the **delete()** method of the file. We don't need to close the stream using the **close()** method because for deleting a file, we neither use the FileWriter class nor the Scanner class.

Let's take an example to understand how we can write data into a file.

DeleteFile.java

```
// Importing the File class
import java.io.File;
class DeleteFile {
  public static void main(String[] args) {
    File f0 = new File("D:FileOperationExample.txt");
    if (f0.delete()) {
      System.out.println(f0.getName()+ " file is deleted successfully.");
    } else {
      System.out.println("Unexpected error found in deletion of the file.");
    }
  }
}
```

Output:

```
C:\Windows\System32\cmd.exe

C:\Users\ajeet\OneDrive\Desktop\programs>javac DeleteFile.java

C:\Users\ajeet\OneDrive\Desktop\programs>java DeleteFile
FileOperationExample.txt file is deleted successfully.

C:\Users\ajeet\OneDrive\Desktop\programs>
```

Explanation:

In the above code, we import the **File** class and create a class **DeleteFile**. In the main() method of the class, we create **f0** object of the file which we want to delete. In the **if** statement, we call the **delete()** method of the file using the f0 object. If the delete() method returns true, we print the success custom message. Otherwise, it jumps to the else section where we print the unsuccessful custom message.